DeployGuard Viva Master Notes

Source basis: committed repository `a571c14e9afc45bedf22ec1e4d22e136e8e3005e` (`test1` worktree was dirty during study, so uncommitted bytes were not used as proof). This is a source study, not AWS end-to-end certification.

1. What DeployGuard is

DeployGuard is a control plane for deploying a GitHub repository to AWS. The frontend lets a developer configure a project and start an operation. The NestJS backend admits that operation, records its immutable inputs in PostgreSQL, dispatches a reusable GitHub Actions workflow, and later reconciles its evidence. The workflow performs the long-running work: Railpack build, image validation, ECR publish, Trivy, Terraform and runtime verification. A release becomes stable only when the backend validates the returned evidence.

Main product flow:

`User → React UI → NestJS → PostgreSQL + GitHub App → GitHub Actions → Railpack build → ECR → Trivy → Terraform/ECS → CloudWatch → backend reconciliation → UI/SNS`.

This distinction is central: GitHub Actions executes; DeployGuard decides whether the operation was admitted and whether the evidence is sufficient to present it as successful.

2. Diagrams

- A. Complete architecture
- B. Deployment sequence
- C. AWS/VPC diagram
- D. Authentication sequence
- E. Database/entity overview
- F. Module dependency map
- G. Failure-handling flow

The checked diagrams include their repository source references. The notes below are the detailed explanation for them.

3. Frontend

The frontend is React 18 + Vite. `frontend/src/main.jsx` starts the app; `AppRoutes.jsx` defines lazy-loaded routes, and `AuthContext.jsx` is the client authentication state.

Surface	Main page/components	What it does
Public	`Landing`, `About`, `GithubConnecting`	Marketing and GitHub OAuth return handling.
Project workspace	`Projects`, `ProjectDetails`	Lists workspace projects; shows live/ current-state summary.

Creation	`NewProject`	Repository/ App installation selection, branch, service directories, env input, database tier, then deploy.
Deployment	`ProjectPipeline`, `ProjectOverviewLifecycle`, `PipelineExecution`, `PipelineGraph`	Polls durable operation history and renders stages, failures, reset/retry/ rollback/ destroy actions.
Project configuration	`ProjectSettings`, `EnvironmentVariablesPanel`, `EnvVarTable`, `NotificationSettingsPanel`	Service-scoped variables, protected ownership, notifications, branch and service settings.
Infrastructure	`ProjectInfrastructure`	Current topology/ evidence, cost estimate and Terraform export.

Monitoring	`ProjectMetrics`	Runtime metrics and CloudWatch application-log stream.
Troubleshooting	`ProjectTroubleshooting`	Starts and reads advisory AI analysis sessions.
Billing	`Billing`	Subscription, usage, checkout, portal and invoice UI.
Administration	`AdminLogin`, `AdminUsers`, `AdminCloudCleanup`	Separate admin session; users, audit, project overview and controlled cleanup.

The HTTP wrapper is `frontend/src/api/client.js`; `projectApi.js` covers projects, deploy lifecycle, configuration, state, storage, orchestration and observability. `platformApi.js` covers AI, notifications, billing and Terraform export. Requests include browser credentials.

`AuthContext` calls `/api/auth/me` initially and again when the window gains focus/visibility. `ProtectedRoute` needs any authenticated developer session. `RoleProtectedRoute` only permits the developer role for `/deploy`. `AdminProtectedRoute` separately calls `/api/admin-auth/me` for the admin console. UI guards improve navigation, but backend guards remain authoritative.

4. Backend and module communication

`backend/src/main.ts` creates NestJS, Helmet, credentialed CORS and a whitelist validation pipe. `AppModule` sets up PostgreSQL TypeORM and attaches `AuthenticatedUserMiddleware` to all routes.

Major module	Purpose	Input → process → output	Important files
Auth + Users	GitHub OAuth, signed sessions, roles	OAuth code/ cookie → GitHub profile + user lookup → signed cookie/user	`auth.service.ts`, `auth.controller.ts`, `user.entity.ts`
Projects	Canonical project configuration and release admission	API project/ deploy request → ownership, source/build/ requirements checks → project/run/ config revisions	`projects.service.ts`, `projects.controller.ts` `railpack-deployment.service.ts`
GitHub integration	App installation tokens, repository reads and workflow dispatch	owner/repo + installation → GitHub REST calls → source facts/ dispatch receipt	`github-app.service.ts`, `pipeline/github-actions.service.ts`

Failure diagnostics	Deterministic owner/code/stage/retry interpretation	workflow evidence → catalog/classifier → persisted diagnosis	`failure-diagnostic.service.ts`, `failure-contract.catalog.ts`
Observability	Read-only CloudWatch logs/metrics and Prometheus export	live release identity → AWS read → API/SSE/Prometheus text	`cloudwatch-logs.service.ts`, `cloudwatch-metrics.service.ts`
Notifications	SNS subscription preferences and deduplicated delivery	lifecycle event → preference + confirmed subscription → SNS publish/delivery row	`notification-dispatcher.service.ts`, `sns-notification.adapter.t`
Billing	Stripe lifecycle and entitlement/usage gate	plan/webhook/usage → normalized account/subscription state → allow/deny/account UI	`billing.service.ts`, `entitlement.service.ts` `project-usage.service.ts`
FinOps	Terraform-plan cost estimate	plan artifact → Infracost → normalized resource costs	`github-actions-cost-evidence.service.ts`, `infracost.service.ts`

AI troubleshooting	Evidence-bound advice	operation + sanitized evidence → Gemini JSON → validated advisory result	`ai-evidence.service.ts` `ai-evidence-preprocessor.service.t` `ai-provider.adapter.ts`
State/storage/ lifecycle	State history, locks, recovery, backups, inventory/ destroy support	project/ current state → safety/ recovery records → read/control APIs	`state-management`, `storage`, `infrastructure` `lifecycle`

`ProjectsModule` is the active orchestration entry surface. It imports audit, users, read-only AWS CLI, notifications and billing. `AppModule` imports Auth, Users, Admin, Audit, Projects, Terraform Export, AI, Observability and Billing. Some state/storage/orchestration/lifecycle modules remain supporting or historical capability modules; do not claim all are direct `AppModule` imports in this commit.

5. Authentication, GitHub App and RBAC

1. User opens `/api/auth/github`. `AuthController.github` creates a random state, stores it in a 10-minute HttpOnly state cookie, audits the start, and redirects to GitHub OAuth.
2. GitHub returns `code` and `state` to `/api/auth/github/callback`. Controller compares state with the cookie.
3. `AuthService.handleGitHubOAuthCallback` exchanges the code, requests profile and emails, then `UsersService.findOrCreate` persists the user and stores the GitHub access token. GitHub users cannot be admin users.
4. The backend creates an HMAC-signed `deploy_guard_session` cookie, 7 days, HttpOnly, `SameSite=Lax`; frontend calls `/api/auth/me`.

5. Middleware verifies signature, expiry and `developer` audience, then loads a non-disabled user into `req.user`. `requireRole` returns 401 if absent and 403 if role is not allowed.

Roles: `admin`, `developer`, `readonly`. Read-only can access viewing routes guarded for all three roles, while mutations normally add Admin/Developer guards. Admin is deliberately a separate email/password, non-GitHub account with an 8-hour `deploy_guard_admin_session`; admin UI uses the separate endpoint/cookie. Audit logging records OAuth start/success/failure, logout, project actions, role changes and notification events.

GitHub OAuth authenticates a human. The GitHub App is different: `GithubAppService` uses an installation to obtain repository-scoped credentials for repository inspection, source resolution and workflow dispatch.

6. Project creation and configuration

`NewProject.jsx` obtains App connection status and repositories, inspects a selected `owner/repository`, lists branches and browses repository directories. `CreateProjectDto` accepts repository URL/full name, target branch, environment name, 1–20 services, optional entrypoint service and visibility. A service is a `ProjectDeployableService` with name, repository-relative directory, optional bounded build-target override and order.

Environment values are stored per project and service in `project_environment_variables`. They have `isSecret`, scope (`build`, `runtime`, `both`), required/protected ownership metadata and an encrypted value. The frontend can bulk paste variables. Database attachment is separate `project_database_tiers`/binding configuration; its aliases are platform-controlled, not arbitrary user environment substitution.

7. One deployment, end to end

1. `ProjectOverviewLifecycle` calls `deployGithubActionsDeployment`, `POST /api/projects/:projectId/deploy`.
2. `ProjectsController.deployGithubActions` calls `RailpackDeploymentService.deploy` and audits accepted requests.
3. `dispatch` calls `admitLifecycleOperation`, persists `ProjectPipelineRun`, immutable configuration and generation context.
4. It obtains a GitHub App token, resolves the exact 40-character source SHA with `RepositorySourceService`, resolves build targets/ports/requirements at that SHA and stores revisions.
5. For normal deploy it calls `ManagedDatabaseReconciliationService.reconcile`; admission only proceeds when its recovery decision permits it. Reset-fresh is only allowed under the existing recovery/reset contract and does not silently delete healthy persistent DB data.
6. It verifies reusable caller/OIDC/AWS capability, materializes runtime secrets, constructs immutable workflow inputs and calls `GithubActionsService.triggerWorkflow`.
7. The reusable workflow checks out the exact application SHA, downloads the pinned Railpack builder contract, builds images, locally validates runtime reachability, pushes immutable ECR images, scans exact digest images with Trivy, then materializes Terraform.
8. Terraform runs `init`, `validate`, `plan` and `apply`; plan failure is `DG_TERRAFORM_PLAN_FAILED` at `terraform_plan`, distinct from apply failure.
9. For DB-attached app service Terraform creates it at desired count 0. The workflow verifies database task readiness; for MySQL it also requires the grant reconciler exit code 0. It calls ECS `UpdateService` to raise that one app service. Non-DB services are not globally blocked.
10. Workflow verifies ECS deployment, stopped-task/container/target evidence and public reachability; it uploads a typed release-result or failure artifact.
11. `RailpackDeploymentService.reconcile` finds the GitHub run. `validatedReleaseEvidence` and `finalizeVerifiedRelease` persist stable release/generation projections. The UI polls current state/history rather than

assuming dispatch means success.

For a certified release-only redeploy, the backend can mark `release_only=true`; workflow uses immutable task-definition/ECS update path. Terraform has `ignore_changes = [desired_count, task_definition]` for application ECS service so later topology applies do not revert release control, while destroy still removes it. Rollback selects an exact stable immutable target; it does not rebuild source.

8. GitHub Actions, builds, image identity and security

The workflow is `.github/workflows/deployguard-reusable.yml`, `workflow_call` rather than an unbounded backend process. It is used because a build/Terraform/ECS wait can exceed an HTTP request, needs isolated ephemeral tools, and GitHub provides run status and artifact storage. Backend tracks the work with `ProjectPipelineRun.githubWorkflowRunId`, immutable dispatch inputs/fingerprint and periodic reconciliation.

Railpack performs the build. The workflow verifies local image ID, pushes to ECR repositories configured immutable, then looks up the digest. The resulting identity is `registry/repository@sha256:<64 hex>`; tags alone are not the final identity.

Trivy scans those exact immutable service images. Findings include service, image, vulnerability ID, severity, package, installed/fixed version, target and inferred origin. The enforced policy blocks fixable `CRITICAL` application-dependency findings (`DG_TRIVY_POLICY_BLOCKED`); scanner inability when enforced gives `DG_TRIVY_SCAN_FAILED`. It writes a bounded security result including counts, evidence and SHA-256 evidence hash, which is carried into release/failure evidence and the UI projection.

9. Cost estimation

The workflow produces immutable Terraform plan artifacts. On completed deploy/rollback reconciliation, `GithubActionsCostEvidenceService.capture` retrieves `deployguard-cost-plan.json` (and optionally project plan), runs official Infracost, normalizes costs into `project_cost_estimates` and `project_cost_resource_breakdowns`, and records a threshold notification when applicable. It only runs automatically when an Infracost API key, workflow run ID, generation and supported action exist. It is evidence after plan generation, not a replacement for Terraform admission. The Infrastructure screen displays it.

10. Terraform, state and recovery

The active generation runtime is `infrastructure/railpack-runtime`: ECS cluster/services/task definitions, per-service ALB/target group/listener/security groups, CloudWatch log groups, optional managed DB ECS service, Secrets Manager, EFS and Cloud Map. The workflow copies this exact control-plane SHA into a temporary runtime Terraform directory. It initializes an S3 backend with key `projects/<project>/<environment>/runtime/terraform.tfstate`, encryption and `use_lockfile=true`: native S3 lockfiles avoid a separate DynamoDB locking dependency.

`backend/terraform/base-network` is a reusable foundation with network, ALB, EFS, Cloud Map, registry, database service and ECS modules. Its network module supports public/private subnets and NAT strategy. Do not confuse it with the active Railpack runtime: the latter takes an existing VPC plus `public_subnet_ids` and sets ECS `assign_public_ip = true`.

State-management entities/services record state snapshots, lock observations, validation and recovery requests. Terraform plan/apply are workflow boundaries; persistent-data reset must respect reconciliation and recovery evidence. Destroy first uses Terraform destroy and then verifies targeted cleanup before local project deletion.

11. AWS architecture

- VPC: supplied to the canonical runtime; resource names/tags are project/operation scoped.
- Public subnets: ALBs use them. In the current `railpack-runtime`, app and optional DB ECS tasks also use these configured subnet IDs with public IP assignment.
- Internet Gateway/NAT: foundation network module creates the routing model; NAT is the private-subnet egress option. Active runtime does not itself select private task subnets.
- ALB: external per-service HTTP ingress, target group IP targets, health check through a sidecar transport probe.
- ECS Fargate: app task has application container plus `deployguard-transport-probe`. Optional DB is a separate ECS service.
- ECR: immutable application image repositories/digests.
- EFS: encrypted optional database persistence through an access point and mount targets.
- Cloud Map: optional managed DB private namespace/service; app gets `database.<project>.internal` aliases.
- CloudWatch: application/DB log groups, ECS CPU/memory and ALB latency/healthy-host metrics.
- SNS: backend-owned email lifecycle notification provider, not Terraform runtime deployment transport.
- S3: Terraform remote state and lockfile backend; the workflow removes exact state object after verified destroy.

12. Monitoring and AI troubleshooting

CloudWatch logs are read via `CloudWatchLogsService`; it first resolves authoritative LIVE release/service identity, filters log events and sanitizes text. The log stream API is server-sent events and emits connection/generation-change/log/heartbeat/warning events. `CloudWatchMetricsService` queries ECS CPU/memory and ALB latency/healthy/unhealthy host metrics, caches briefly and derives runtime availability. `AwsPrometheusExportService` exposes latest

points at `/api/monitoring/metrics`; `monitoring/prometheus.yml` scrapes it and Grafana is provisioned with Prometheus.

AI is one advisory module. `AiEvidenceService` gathers operation-correlated pipeline, observability and release data. `AiEvidencePreprocessorService` sanitizes, limits and references it. `AiProviderAdapter` calls configured Google Gemini with JSON schema and an instruction to use only supplied evidence. Its validator rejects malformed output, unknown evidence references and answers that contradict authoritative owner/root-cause/retry/problem-type facts. It stores sessions/messages/results. It cannot dispatch workflows, call Terraform, write AWS or alter deployment state.

13. Notifications, billing and destroy

Notifications: project owner configures preferences and an email subscription. `SnsNotificationAdapter` creates/fetches SNS topic/subscription; email must be confirmed. `NotificationDispatcherService` classifies lifecycle events, deduplicates them, persists delivery attempt/status and publishes with up to three short retries. Examples: deploy/rollback/destroy started/completed/failed, security block, unhealthy runtime and cost threshold.

Billing: `BillingService` exposes summary, Stripe Checkout, customer portal, signed webhook handling and invoices. Tables cover account, subscription, checkout, invoice, payment, webhook event, usage counter/event. `EntitlementService` and `ProjectUsageService` turn this durable commercial state into admission/usage decisions. Stripe is not part of the AWS deployment executor.

Destroy: developer must type the configured destroy confirmation phrase. `RailpackDeploymentService.destroy` dispatches the immutable destroy workflow. It runs Terraform destroy, removes precisely checked DeployGuard-owned runtime secrets/ECR repositories/state artifact, creates a `deployguard.destroy-result/v2` verification result, and only then

`ProjectDeletionService.finalize` deletes cloud-associated control-plane rows/project. If cloud destruction is verified but local deletion fails, retry finalizes locally without redispatching Terraform. Database data is not deleted merely to make a deployment fresh.

14. PostgreSQL tables grouped by purpose

Group	Important tables/entities	S info
Identity/audit	`users`, `github_app_installations`, `audit_logs`, `project_user_activity`	User role/s App i meta secu histo viewe
Project configuration	`projects`, `project_deployable_services`, `project_environment_variables`, `project_database_tiers`, `project_service_bindings`, `project_environment_routes`	Repo branc envir servi direc ports encry varia DB/ru bindi
Immutable release facts	`project_pipeline_runs`, `project_pipeline_events`, `project_deployment_generations`, `project_configuration_snapshots`, `project_build_target_revisions`, `project_service_runtime_config_revisions`, `project_generation_service_revisions`	Oper SHA, run, s stage exact build/ histo

Runtime and safety	`project_stable_releases`, `project_deployments`, `project_rollback_records`, `project_terraform_states`, `project_terraform_locks`, `project_state_validation_results`, `project_state_recovery_requests`, `project_deployment_queue_items`	Stabl lifecy histo remo evide and r
Infrastructure/ storage	`project_infrastructure_environments/ events`, `project_deployment_readiness_snapshots`, `project_service_discovery_records`, `project_persistent_storage`, `project_backup_records`, `project_storage_events/restore_requests`	Reso readi servi disco persi back
Observability/ AI	`project_observability_events`, `project_stage_metrics`, `project_pipeline_metric_summaries`, `project_runtime_metric_snapshots`, `project_log_stream_sessions`, `ai_analysis_sessions/messages/results`	Meas obse telem evide boun troub conv
FinOps/ billing/ notification	`project_cost_estimates/breakdowns/ settings`, `billing_*`, `notification_preferences/subscriptions/ deliveries`	Cost plans paym usag notifi state
Operator cleanup/ export	`project_cloud_states`, `central_cloud_resources`, inventory/cleanup/ destroy challenge tables, `terraform_export_artifacts`	Contr admi clean trace Terra expo

The central orchestration table is `project_pipeline_runs`: it records project, triggerer, repo/branch/SHA, GitHub run identity, status, current

stage, timestamps, ECR fields, failure owner/provider/code/service and immutable JSON metadata.

15. Failure handling: detector → storage → UI

Failure	Detector	Durable storage
Build/ application validation	Railpack/local validation and terminal structured failure artifact	pipeline run + events + failure
Trivy	workflow scan of digest-pinned images	security result/failure artifact – metadata/diagnostic
Terraform plan	workflow command explicitly emits `DG_TERRAFORM_PLAN_FAILED`, stage `terraform_plan`	artifact → `ProjectPipelineRun.fai currentStage`
Terraform apply/ AWS	workflow apply boundary and AWS verification	failure artifact + classifier
ECS/ALB/ runtime	runtime verifier inspects ECS events, stopped task/container exits, target health and public reachability	AWS verification/failure artifac events/release projection
GitHub	dispatch receipt/reconciliation/ API failure	operation dispatch state and fa diagnostic

Failure owners are `REPOSITORY_APPLICATION`, `DEPLOYGUARD_PLATFORM`, `EXTERNAL_PROVIDER` (for example AWS/ GitHub/Railpack) and `UNVERIFIED`. The UI should never present a confident invented root cause when evidence is ambiguous.

16. Important design decisions

1. **GitHub Actions for long work:** build, Docker, security scanner, Terraform and ECS stability can be slow and need isolated tools. An async workflow avoids holding an API request open; operation ID plus artifact/reconciliation makes it trackable.
2. **PostgreSQL for durable orchestration:** browser refreshes and backend restarts must not erase release identity, immutable inputs, statuses, evidence, audit or recovery conditions.
3. **S3 native locking:** the Terraform S3 backend has `use_lockfile=true`; it makes the remote state object and lock behavior native to the state store and avoids parallel Terraform mutation.
4. **AI is advisory only:** infrastructure mutation is behind authenticated backend admission and immutable workflow contracts. AI sees sanitized evidence, has schema/grounding checks, and does not receive control-plane mutation capability.
5. **Terraform vs ECS ownership:** Terraform creates/deletes resource topology. DeployGuard/ECS handles post-bootstrap desired count and immutable task-definition release changes; `ignore_changes` prevents later topology apply from fighting that runtime decision.

17. Fast viva answers

- **What is the single source of truth for a deployment?** The PostgreSQL operation/generation/release records, reconciled against exact GitHub workflow artifacts; not a transient frontend state.
- **What proves the image deployed?** Digest identity from ECR, stored in immutable workflow/release evidence, not a mutable tag.
- **Does a database failure stop every service?** No. The DB-attached service is gated; unrelated application services remain independent.
- **Why do you need both OAuth and a GitHub App?** OAuth identifies the user; GitHub App installation credentials authorize

repository/workflow operations.

- **Can an admin sign in with GitHub?** No. The source enforces separate non-GitHub admin credentials and an admin session audience.
- **What happens after GitHub Actions says success?** Backend validates typed release evidence and then persists/promotes stable release; success alone is not enough.
- **Does cost estimate change Terraform?** No. It derives displayable evidence from immutable plan artifacts.
- **Is the database a managed RDS service?** No. In the canonical runtime it is an optional PostgreSQL/MySQL/MongoDB containerized ECS service with EFS persistence, Secrets Manager and Cloud Map.

Evidence index

Primary code: `backend/src/projects/railpack-deployment.service.ts`, `backend/src/projects/pipeline/github-actions.service.ts`, `.github/workflows/deployguard-reusable.yml`, `infrastructure/railpack-runtime/main.tf`, `backend/src/app.module.ts`, `frontend/src/routes/AppRoutes.jsx`, `backend/src/auth/auth.service.ts`, `backend/src/observability/*`, `backend/src/notifications/*`, `backend/src/billing/*`, `backend/src/ai-troubleshooting/*`.